

**C.B PATEL COMPUTER COLLEGE &
J. N. M. PATEL SCIENCE COLLEGE, SURAT *VEER*
NARMAD SOUTH GUJARAT UNIVERSITY,
*SURAT***



**A
PROJECT REPORT
ON
*The Donor***

**AS PARTIAL REQUIREMENT FOR THE DEGREE OF
BACHELOR OF COMPUTER APPLICATION
[B.C.A]
YEAR – 2025-26**

SUBMITTED BY:

**Mali Rajesh
Gupta Raj
Javiya Manav
Munde Rujal**

GUIDED By:

Mrs . Jenish Bhavsar

ACKNOWLEDGEMENT

Mrs. Jenish Bhavsar, our professor and project coordinator, has been very prudent to us throughout our college studies. She is the person who has been giving direction to our work and the shape to our imagination. We express our regards to her from the core of our heart.

We also like to thank **Dr. Faiyazalam Shaikh** and **H. O. D Dhananjay Sir** and all the professors who are always ready to give best guidance. They are the individuals who give solutions whenever required.

We would also like to acknowledge all our friends and colleagues, team members for their help and encouragement from time to time. The constant support and encouragement of my friend deeply appreciates. The project indeed gave challenging and exhilarating experience in designing and developing the required system.

Finally we would like to thank our parents for their support throughout the project. We owe a special debt to our family & friends for their supports blessing and encouragement for us.

THANKS ALL.....

Sr No.	Topic	Page No.
1.	Introduction	
	1.1 Executive Summary	5
	1.2 Project Profile	6
2.	Objective of Project	
	2.1 Problem Statement	8
	2.2 Module of the System	9
3.	Technology Stack	
	3.1 Frontend	15
	3.2 Backend	17
	3.3 Infrastructure & Deployment	
4.	System Architecture	19
	4.1 Architecture Overview	21
	4.2 Project File Structure	22
5.	Features & Modules	31
	5.1 Landing Page (Public)	33
	5.2 Authentication System	41

	5.3 Menu Page (Customer)	45
	5.4 Admin Dashboard	15
6.	API Documentation	16
	6.1 Authentication	16
	6.2 Menu	16
	6.3 Orders	16
	6.4 Payments	17
	6.5 Loyalty	17
7.	User Flow & Journey	19
	7.1 Customer Journey	19

1.1 Executive Summary

Annapurna Smart Canteen is a production-grade, full-stack web application designed to digitise and streamline the entire food ordering workflow of a college campus canteen. The system eliminates long queues, manual billing errors, and paper-based order tracking by providing a seamless digital experience for both students and canteen staff.

The platform allows students to browse the complete menu, add items to a persistent cart without requiring login, and complete checkout securely via UPI payment (Razorpay). On the operations side, canteen administrators receive orders in real time on a live dashboard, manage the menu, track revenue, and update order statuses through a dedicated control panel.

Student Experience	Admin Operations
• Browse full menu freely — no login needed • Add items to cart, explore the site • Login only required at checkout • Cart items preserved after login • UPI & Cash on Delivery payment • Real-time order status tracking • Loyalty points and tier rewards • Order history and receipts	• Live orders dashboard (auto-refreshes) • 5 KPI cards: items, orders, revenue • One-click status updates (Pending → Preparing → Ready → Delivered) • Full menu management (add / edit / delete) • Toggle item availability in real time • Food DNA nutritional data per dish • Dark mode across entire platform • Responsive on desktop & mobile

1.2 PROJECT PROFILE

Project Name	Annapurna Smart Canteen
Type	Full-Stack Web Application (SPA)
Domain	Food Technology / Campus Management / EdTech
Live URL	annapurna-smart-canteen.netlify.app
Backend URL	annapurna-smart-canteen1.onrender.com
Deployment	Netlify (Frontend) + Render (Backend)
Year	2025
Internal guide	Mrs . Jenish Bhavsar
Submitted To	C.B Patel Computer College
Developed By	Mali Rajesh Gupta Raj Javiya Manav Munde Rujal

2. Problem Statement

College canteens face several operational challenges that a digital system can solve:

Problem (Manual System)	Solution (Annapurna)
Long queues during peak hours (lunch break)	Pre-ordering from class; food ready on arrival
Manual billing errors and change issues	Digital UPI payments; exact amounts, no change
No visibility on order status for students	Live order tracking: Pending → Preparing → Ready
Paper-based sales records; no analytics	Admin KPI dashboard with revenue tracking
No loyalty or engagement mechanism	Loyalty points system: Bronze → Silver → Gold

3. Technology Stack

The application is built using modern, industry-standard technologies across the full stack.

3.1 Frontend

Category	Technology	Purpose
UI Framework	React.js 18	Component-based single-page application
Routing	React Router v6	Client-side navigation, protected routes
HTTP Client	Axios	API calls to Express backend
Styling	CSS-in-JS	Scoped styles via template literals, dark mode
Auth (Client)	Firebase Auth SDK	Login state, token management
Fonts	Google Fonts	Playfair Display + DM Sans + Space Mono + Syne
Animation	CSS Keyframes	Page transitions, micro-interactions
Build Tool	Create React App	Webpack bundling, dev server, production build

3.2 Backend

Category	Technology	Purpose
Runtime	<code>Node.js</code>	JavaScript server runtime
Framework	<code>Express.js</code>	REST API routing and middleware
Database	<code>Firestore</code>	NoSQL cloud database (Firebase)
Auth (Server)	<code>Firebase Admin SDK</code>	Token verification, user management
Payments	<code>Razorpay</code>	UPI payment links, webhooks
Google Auth	<code>google-auth-library</code>	Verify Google OAuth tokens
Security	<code>crypto (Node)</code>	Webhook signature verification (HMAC-SHA256)
Config	<code>dotenv</code>	Environment variable management

3.3 Infrastructure & Deployment

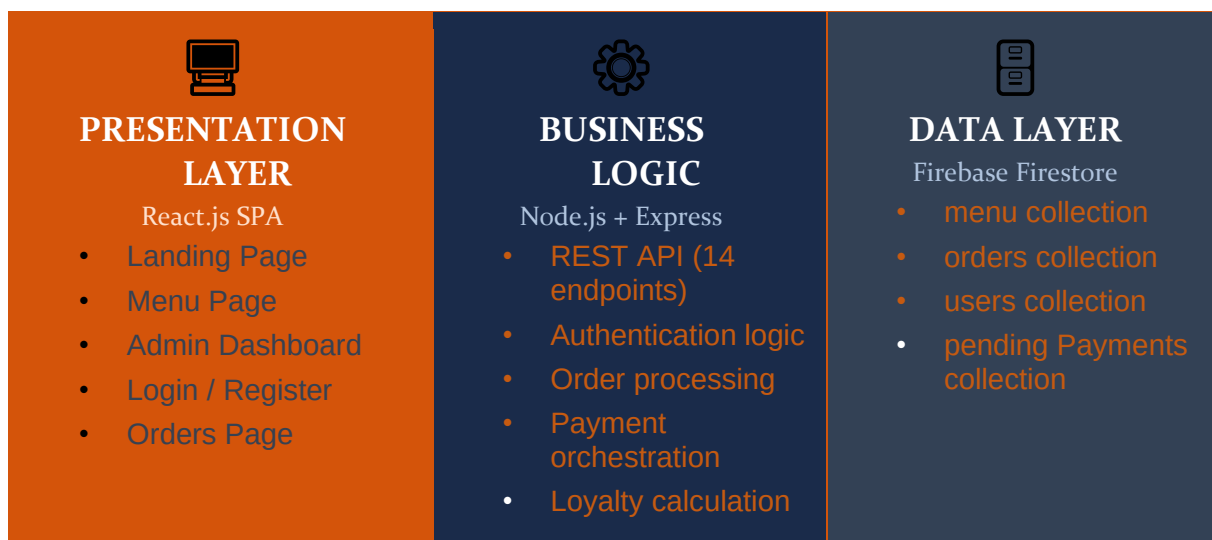
Service	Platform	Details
Frontend Hosting	<code>Netlify</code>	Auto-deploy from Git, global CDN, custom domain
Backend Hosting	<code>Render</code>	Always-on Node.js service, env var management

Service	Platform	Details
Auth Provider	Firestore Auth	Email/Password + Google OAuth
Database	Firestore	Cloud NoSQL, real-time, serverless
Payments	Razorpay	Indian payment gateway, UPI support
Version Control	Git + GitHub	Source code management, CI/CD triggers

4. System Architecture

The application follows a clean three-tier architecture separating presentation, business logic, and data layers.

4.1 Architecture Overview



4.2 Project File Structure

Frontend (src/) App.js Route definitions & auth state pages/LandingPage.jsx Public landing page + guest cart pages/Login.jsx Animated login with Google OAuth pages/Register.jsx Futuristic register UI pages/MenuPage.jsx Main menu with cart, DNA panel	Backend (backend/) server.js All 14 API routes + logic .env (secret – never commit) Firebase keys, Razorpay keys Deployment Files public/_redirects Netlify SPA routing fix Database Collections Firestore: menu
--	--

pages/AdminDashboard.js
Live orders + menu management
pages/OrdersPage.js
Order history for customers
firebase.js
Firebase app initialisation

Dishes with DNA stats
Firestore: orders
Customer orders
Firestore: users
User profiles + loyalty points
Firestore: pendingPayments
Active Razorpay payment links

5. Features & Modules

5.1 Landing Page (Public)

The landing page is accessible to all visitors without login. It showcases the canteen brand, featured dishes, and provides a seamless entry into the ordering flow.

	Feature	Description	Status
	Hero Section	Full-screen animated hero with CTA buttons and food background	Implemented
	Menu Showcase	11 dishes displayed with images, ratings, price, and prep time	Implemented
	Guest Cart	Add items to cart WITHOUT login — cart persists in localStorage	Implemented
	Qty Controls	Inline +/- quantity controls per dish card on landing page	Implemented
	Cart Badge	Orange badge in navbar showing item count when cart has items	Implemented
	Cart Drawer	Slide-in cart drawer with item list, total, and checkout button	Implemented
	Smart Login Gate	Login prompt only appears when tapping "Login & Checkout"	Implemented

	Feature	Description	Status
	Cart Persistence	Cart items restored automatically after login in MenuPage	Implemented
	Testimonials	Auto-advancing carousel with student testimonials	Implemented
	Restaurants	Featured restaurant/stall cards with ratings	Implemented
	Mobile Responsive	Hamburger menu, responsive grid, touch-friendly	Implemented

5.2 Authentication System

	Feature	Description	Status
	Email/Password	Register and login with email and password via Firebase Auth	Implemented
	Google OAuth	One-click Google Sign-In via @react-oauth/google	Implemented
	Protected Routes	Menu, Orders, Admin pages require authentication	Implemented
	Atomic Auth State	Single setState for user+isAdmin+loading — eliminates race conditions	Implemented

	Feature	Description	Status
	Admin Access	Admin flag stored in Firestore users collection	Implemented
	Animated Login	Cinematic letter-by-letter animation on Login page	Implemented
	Futuristic Register	Dark theme with grid background, glowing rings, Space Mono font	Implemented

5.3 Menu Page (Customer)

	Feature	Description	Status
	Search & Filter	Search by name, filter by category (Breakfast/Lunch/Snacks/etc)	Implemented
	Cart System	Add/remove items, update quantities, persistent cart in localStorage	Implemented
	Food DNA Panel	Slide-in panel with 5 nutritional stats per dish (animated bars)	Implemented
	Checkout Flow	Delivery name, location input, payment method selection	Implemented
	UPI Payment	Razorpay payment link, QR code, real-time payment status polling	Implemented

	Feature	Description	Status
	Cash on Delivery	COD option with instant order confirmation	Implemented
	Dark Mode	Full dark/light mode toggle persisted in localStorage	Implemented
	Loyalty Points	Points earned per order, Bronze/Silver/Gold tier system	Implemented
	Order Receipt	Full receipt modal after successful payment	Implemented
	ETA Calculator	Estimated preparation time based on cart items	Implemented

5.4 Admin Dashboard

	Feature	Description	Status
	5 KPI Cards	Menu Items / Live Orders / Preparing / Ready / Total Revenue	Implemented
	Live Orders Feed	Real-time order list, auto-refreshes every 12 seconds	Implemented
	Status Updates	One-click: Pending → Preparing → Ready → Delivered	Implemented
	Mark Complete	Remove order from dashboard once delivered	Implemented
	Add Menu Item	Form to add new dish with image URL,	Implemented

	Feature	Description	Status
		price, category, DNA stats	
	Edit Menu Item	Edit existing dish details and Food DNA stats inline	Implemented
	Delete Item	Remove dish from Firestore and hide from customers	Implemented
	Toggle Availability	Mark items in/out of stock instantly	Implemented
	Dark Mode	Admin dashboard has independent dark mode support	Implemented

6. API Documentation

The backend exposes a RESTful API with 14 endpoints. Base URL: <https://annapurna-smart-canteen1.onrender.com>

6.1 Authentication

Method	Endpoint	Description
POST	/api/auth/google	Verify Google credential, create/update Firebase user, return custom token

6.2 Menu

Method	Endpoint	Description
GET	/api/menu	Fetch all menu items from Firestore
POST	/api/menu	Add new dish (admin only)
PUT	/api/menu/:id	Update dish details and Food DNA stats
PUT	/api/menu/:id/stock	Toggle in-stock / out-of-stock status
DELETE	/api/menu/:id	Remove dish from menu permanently

6.3 Orders

Method	Endpoint	Description
POST	/api/orders	Create new order (COD or UPI)
GET	/api/orders	Fetch all orders (admin dashboard)
GET	/api/orders/user/:userId	Fetch orders for specific customer
PUT	/api/orders/:id/status	Update order status (admin action)
DELETE	/api/orders/:id	Delete completed/cancelled order

6.4 Payments

Method	Endpoint	Description
POST	/api/create-payment-qr	Create Razorpay payment link, save to Firestore
GET	/api/payment-status/:qrId	Poll payment status (webhook + API fallback)
POST	/api/razorpay-webhook	Receive payment confirmation from Razorpay

6.5 Loyalty

Method	Endpoint	Description
GET	/api/loyalty/:userId	Get user loyalty points and tier (Bronze/Silver/Gold)
POST	/api/loyalty/:userId/add	Increment loyalty points after successful order
GET	/health	Server health check — verify all env vars are set

7. User Flow & Journey

7.1 Customer Journey

Step	Action	Details
1	Visit	Opens annapurna-smart-canteen.netlify.app → Sees the Landing Page with full menu, restaurants section, and features
2	Browse & Add	Taps + on any dish → Inline qty controls appear → Cart badge shows in navbar → Cart drawer slides in from right
3	Checkout Tap	Taps "Login & Checkout" → Redirected to Login page. Cart is saved in localStorage (not lost)
4	Login	Signs in with Email or Google → Redirected to /menu → Cart automatically restored from localStorage (no re-adding needed)
5	Place Order	Enters delivery name & location → Selects UPI or COD → Confirms order → Order saved to Firestore
6	Track Order	Sees live status: Pending → Preparing → Ready → Delivered. Loyalty points credited automatically on completion

7.2 Routing Logic

Route	Access	Behaviour
/	Public	Landing page. If logged in → redirect to /menu
/landing	Public (always)	Landing page accessible even when logged in
/login /register	Public	If already logged in → redirect to /menu
/menu /orders	Protected (login)	Requires Firebase auth. If not logged in → /login
/admin	Protected (login only)	Accessible to any logged-in user via direct URL

8. Database Schema(Firestore)

The application uses Firebase Firestore — a NoSQL cloud database. Data is organised into four collections.

8.1 menu collection

Field	Type	Description
id	string (auto)	Firestore auto-generated document ID
name	string	Dish name e.g. "Masala Dosa"
price	number	Price in INR e.g. 60
category	string	Breakfast / Lunch / Snacks / Beverages / Desserts
imageUrl	string	Full URL to dish image
description	string	Short dish description
prepTime	number	Preparation time in minutes
available	boolean	Whether dish is available for ordering
inStock	boolean	Whether dish is currently in stock
calories	number	Food DNA — caloric value (0-100 scale)
spice	number	Food DNA — spice level (0-100 scale)
protein	number	Food DNA — protein content (0-100 scale)
popularity	number	Food DNA — popularity score (0-100 scale)
freshness	number	Food DNA — freshness rating (0-100 scale)

Field	Type	Description
createdAt	timestamp	Firestore server timestamp when dish was added

8.2 orders collection

Field	Type	Description
id	string (auto)	Firestore document ID (also order ID)
userId	string	Firebase Auth UID of the customer
items	array	Array of {foodId, name, price, quantity}
total	number	Total amount in INR
status	string	Pending / Preparing / Ready / Delivered
paymentMethod	string	"upi" or "cod"
paymentStatus	string	"pending" or "paid"
razorpayLinkId	string null	Razorpay payment link ID (UPI orders only)
deliveryName	string	Customer name for delivery
deliveryLocation	string	Delivery location (room/table)
eta	string	Estimated preparation time
pointsEarned	number	Loyalty points earned from this order
timestamp	timestamp	Order creation time (Firestore server timestamp)

9. Payment Integration (Razorpay upi)

The payment system uses Razorpay Payment Links to handle UPI transactions. It implements a two-path verification system to ensure reliability.

9.1 UPI Payment Flow

Step	Actor	Action
1	Customer	Selects UPI payment at checkout → frontend calls POST /api/create-payment-qr
2	Server	Creates Razorpay payment link (amount × 100 paise) → saves to Firestore pendingPayments
3	Server	Returns {qrId, paymentUrl} to frontend
4	Frontend	Opens payment URL in new tab → shows QR code + polls payment status every 3s
5	Razorpay	Customer completes UPI payment → Razorpay triggers webhook to /api/razorpay-webhook
6	Server	Webhook verifies HMAC signature → updates Firestore pendingPayments.status = "paid"
7	Frontend	Polling detects status=paid (fast path via Firestore) → shows success + receipt
8	Fallback	If webhook delayed → frontend polls Razorpay API directly (slow path) for confirmation

10.Key Technical Decisions & Bug Fixes

Several important engineering decisions were made during development that are worth documenting.

10.1 Guest Cart Without Login

Decision: Allow customers to browse and add items to cart without authentication. Login is only required at checkout.

Implementation: Cart stored in localStorage under key `annapurna_guest_cart`. When user logs in, `MenuPage` matches guest cart items to real Firestore menu by name (case-insensitive), rebuilds cart with proper IDs, merges with any existing cart, and clears the guest cart key.

Benefit: Removes friction — a customer can explore the full menu and only commit to login when ready to pay. This significantly improves conversion rate.

10.2 Atomic Auth State (React Race Condition Fix)

Problem: The original `App.js` had three separate `setState` calls — `setUser()`, `setIsAdmin()`, and `setLoading()` — across an `async/await` boundary. React could process them in separate renders, creating a window where `loading=false` AND `isAdmin=false` simultaneously. This caused the `AdminRoute` to redirect admins to `/menu` before the Firestore check completed.

Fix: All three values merged into a single state object. A single `setAutho()` call ensures all three values update atomically in one render — making the race condition structurally impossible.

10.3 Firestore OrderBy Bug

Problem: The original GET /api/orders used `.orderBy("timestamp", "desc")`. Firestore silently excludes documents missing the ordered field — so old/test orders without a timestamp field disappeared from the admin dashboard entirely.

Fix: Removed `.orderBy()` entirely. Fetch all documents, normalise timestamps in JavaScript using a `tsToMs()` helper, then sort in JavaScript. This is bulletproof regardless of whether timestamp field exists.

10.4 UPI Order Status Bug

Problem: UPI orders were saved with `status:"confirmed"` instead of `"Pending"`. The admin dashboard only shows orders in `ACTIVE_STATUSES = ["Pending", "Preparing", "Ready"]`, so UPI orders were invisible to the admin.

Fix: Frontend saves all orders with `status:"Pending"`. Backend `normStatus()` function maps `"confirmed" → "Pending"` for auto-healing existing Firestore documents.

10.5 FIREBASE_PRIVATE_KEY Format

Problem: Render sometimes stores environment variables with literal `\n` strings, sometimes with actual newlines. The original code only handled one format, causing Firebase to crash with `"invalid private key"` on deployment.

Fix: The key parsing now detects which format was used and handles both: if the key contains literal `\n` it replaces them; if it already has real newlines it uses as-is.

11. Deployment Guide

11.1 Frontend — Netlify

Step	Action
1	Run <code>npm run build</code> in the frontend directory to create the production build folder
2	Go to netlify.com → "Add new site" → "Deploy manually" → drag and drop the build/ folder
3	OR connect GitHub repo → Netlify auto-deploys on every git push to main branch
4	CRITICAL: Add the <code>_redirects</code> file to <code>public/</code> folder containing: <code>/* /index.html 200</code>
5	This file tells Netlify to serve <code>index.html</code> for all routes, letting React Router handle navigation
6	Without <code>_redirects</code> : any page refresh or direct URL visit (e.g. <code>/menu</code>) returns 404 error

11.2 Backend — Render

Step	Action
1	Go to dashboard.render.com → New → Web Service → connect your GitHub repository
2	Set Build Command: <code>npm install</code> Start Command: <code>node server.js</code>
3	Go to Environment tab and add all required environment variables (see table below)

Step	Action
4	Click Save → Render auto-deploys. Visit /health endpoint to verify all env vars are set correctly

11.3 Required Environment Variables

Variable	Where to find	Notes
FIREBASE_PROJECT_ID	Firebase Console	Project Settings → General → Project ID
FIREBASE_CLIENT_EMAIL	Firebase Console	Project Settings → Service Accounts → Generate Key → client_email
FIREBASE_PRIVATE_KEY	Firebase Console	Project Settings → Service Accounts → Generate Key → private_key (paste full key)
RAZORPAY_KEY_ID	Razorpay Dashboard	Settings → API Keys → Key ID
RAZORPAY_KEY_SECRET	Razorpay Dashboard	Settings → API Keys → Key Secret
RAZORPAY_WEBHOOK_SECRET	Razorpay Dashboard	Webhooks → your webhook → Secret
GOOGLE_CLIENT_ID	Google Cloud Console	Credentials → OAuth 2.0 Client ID

12.Future Enhancements

The following features are planned for future versions of the system:

	Feature	Description	Priority
	Push Notifications	WhatsApp/SMS alerts when order status changes	High
	Dish Ratings	Students can rate and review dishes after ordering	High
	Pre-order Scheduling	Schedule orders in advance for a specific pickup time	Medium
	Revenue Analytics	Charts for daily/weekly/monthly sales trends	Medium
	Promo Codes	Discount codes and offers for special occasions	Medium
	Group Orders	Multiple students can add to a shared cart	Low
	Image Upload	Admin uploads dish photos directly instead of URL	Low
	Multi-language	Hindi and regional language support	Low

Fire Base Schema

(default)	orders	DlfdKQ2cZwzSzzoYSbxC
+ Start collection	+ Add document	+ Start collection
menu	03cDx8UNqyL1r0hjaLuT	+ Add field
orders	BT4b0CgXXbcGPWJrtJh6	deliveryLocation: "SSADHO SCALE 2ND FLOOR"
pendingPayments	DlfdKQ2cZwzSzzoYSbxC	deliveryName: "SAADHO Scale"
users	EtTgyKL4sVcVL8uUR4vf	eta: "18-23 mins"
	KTILHzruQ7r8SFnt2AmT	items
	YdMFcfbvIoOsXANbDc2W	0
	x9NRaN8q95hTgBHNSXqn	foodId: "Unfd44ZDsbP2Yxa5GY0r"
		name: "Butter Nan"
		price: 49
		quantity: 1
		paymentMethod: "upi"
		paymentStatus: "paid"
		pointsEarned: 0

users > BUJCs3f4gAYw...		
(default)	users	BUJCs3f4gAYwAD7I07Dh7iWGwc83
+ Start collection	+ Add document	+ Start collection
menu	2E6lIqz11aVimc09gnryHDscU0...	+ Add field
orders	AZtQPdip6Ra9y2WjMbPLa5NDZM...	email: "Ishamali7373@gmail.com"
pendingPayments	BUJCs3f4gAYwAD7I07Dh7iWGwc...	role: "student"
users	google_1033312733058012712...	
	google_1089263846624695917...	
	google_1099644176642429379...	
	google_1104348196537024500...	
	google_1110572437369862510...	
	google_1131230336209957230...	
	google_1135222072850724740...	
	google_1148949431593654002...	
	google_1180814303409075539...	
	hLdFv0uvRYQ4vFsS2N0ReEXTyP...	
	ir7HWnA++Dgvcv7A9+06rTi844	

Authentication

Users Sign-in method Templates Usage Settings Extensions

 The following authentication features will stop working when Firebase Dynamic Links shuts down soon: email link authentication for mobile apps, as well as Cordova OAuth support for web apps.

 Search by email address, phone number or user UID

Add user



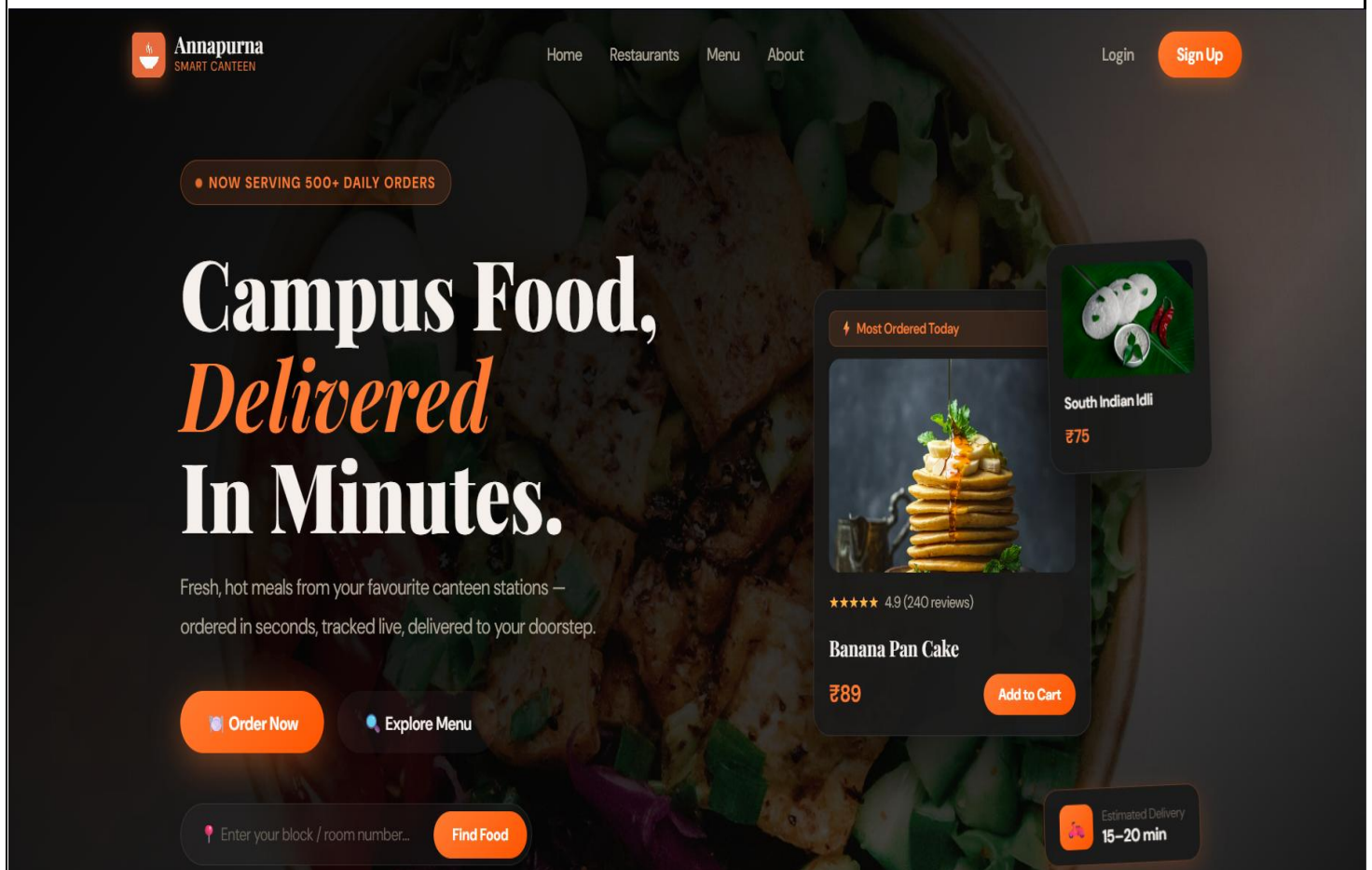
Identifier	Providers	Created ↓	Signed In	User UID
gaurav@gmail.com		17 Mar 2026	18 Mar 2026	pGLOqOMJ2xMYGEIBoR0vX4...
niteshgehl0691785@g...		5 Mar 2026	5 Mar 2026	google_113123033620995723...
jenishbhavsar.cbpc@...		2 Mar 2026	2 Mar 2026	google_113522207285072474...
nitindave120@gmail.com		1 Mar 2026	1 Mar 2026	google_103331273305801271...
sahilvakahariya@gmail....		28 Feb 2026	28 Feb 2026	google_11808143034090755...
rujalmund000@gmail....		28 Feb 2026	28 Feb 2026	google_110434819653702450...
ok@gmail.com		26 Feb 2026	26 Feb 2026	IP9jz3ehUAYP3B59SufGhbKri...
abc@gmail.com		26 Feb 2026	26 Feb 2026	c3WbWTWqmSezsBka0xi45z...

The screenshot shows the Google Cloud Firestore console. The left sidebar displays the project hierarchy: 'menu' > 'TB1821p0a4QJ...'. The main panel is divided into three sections. The top section shows the collection 'menu' with a document 'TB1821p0a4QJQXddK15m' selected. The middle section shows the document's fields: 'available', 'calories', 'category', 'description', 'freshness', 'imageUrl', 'inStock', 'name', 'popularity', 'prepTime', 'price', and 'protein'. The bottom section shows the document's data in a table format.

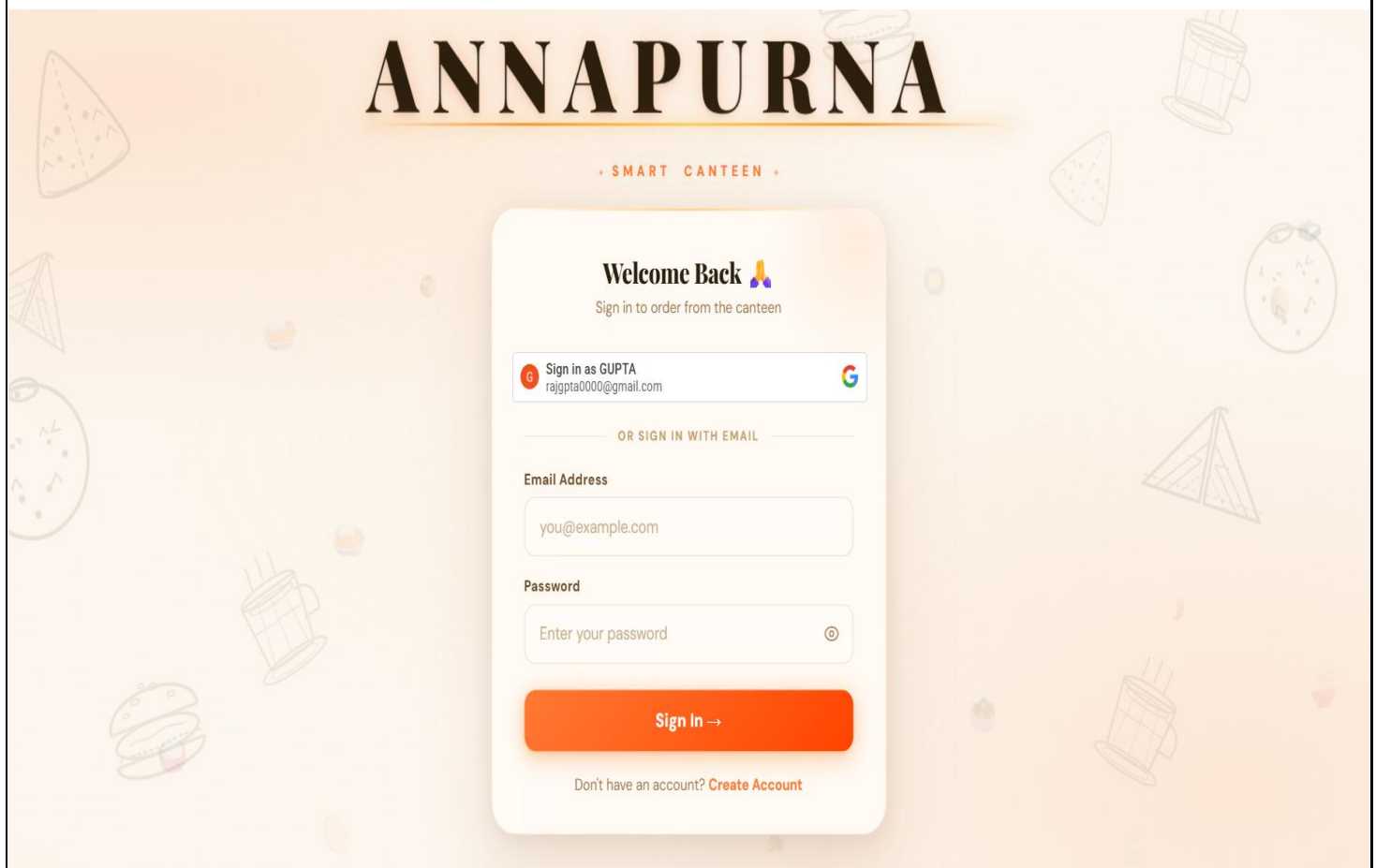
Field	Value
available	true
calories	600
category	"Snacks"
description	"eak number munchio soup"
freshness	7
imageUrl	"https://i.ibb.co/1F3djWy/manchow-soup.jpg"
inStock	true
name	"Manchow Soup"
popularity	99
prepTime	14
price	45
protein	4

13. Output

13.1 Landing page



13.2 Login page



The login page features a light orange background with faint illustrations of food items like burgers, pizzas, and drinks. At the top center, the word "ANNAPURNA" is written in a large, bold, black serif font, with "• SMART CANTEN •" in a smaller orange font below it. The central login form is a white rounded rectangle. It starts with "Welcome Back" and a person icon, followed by the instruction "Sign in to order from the canteen". Below this is a "Sign in as GUPTA" button with a red profile icon and the email "rajgpta0000@gmail.com", accompanied by a Google logo. A separator line with "OR SIGN IN WITH EMAIL" follows. The "Email Address" field contains "you@example.com". The "Password" field is labeled "Enter your password" and includes a toggle icon. A large orange "Sign In →" button is at the bottom of the form, with a link "Don't have an account? Create Account" below it.

ANNAPURNA

• SMART CANTEN •

Welcome Back 👤
Sign in to order from the canteen

 Sign in as GUPTA
rajgpta0000@gmail.com 

OR SIGN IN WITH EMAIL

Email Address

you@example.com

Password

Enter your password 

Sign In →

Don't have an account? [Create Account](#)

13.3 Home Page



Welcome back
rajgta0000

50 pts
Bronze

11 Menu

Orders

Settings

Cart

Bronze Member

50
Loyalty Points

50 pts200 pts → next tier

Earn 10 pts for every ₹100 spent
200 pts → Silver500 pts → Gold

FRESH & VEGETARIAN

Today's Menu

Freshly prepared Indian dishes, made with love every day

48

Breakfast

Lunch

Snacks

Beverages

Desserts

All Dishes (16)

BEVERAGES

Ice Teee

Healthy ice which refresh your mind

<10 min prep

+ Add to Cart

₹100.00

SNACKS

Manchow Soup

ask number munchos soup

<10 min prep

+ Add to Cart

₹45.00

LUNCH

Butter Nan

A freshly prepared vegetarian delight.

<15 min prep

+ Add to Cart

₹48.00

BREAKFAST

Idli (3 pices)

A freshly prepared vegetarian delight.

<5 min prep

+ Add to Cart

₹40.00

LUNCH

Noodles

A freshly prepared vegetarian delight.

<15 min prep

+ Add to Cart

₹60.00

BREAKFAST

Masala Papad

A freshly prepared vegetarian delight.

<10 min prep

+ Add to Cart

₹60.00

OTHER

Dahi Vada

Taste of Canteen

<5 min prep

+ Add to Cart

₹90.00

BEVERAGES

chas

A freshly prepared vegetarian delight.

<5 min prep

+ Add to Cart

₹15.00

DESSERTS

Gulab Jamun

A freshly prepared vegetarian delight.

<10 min prep

+ Add to Cart

₹40.00

OTHER

Ice Cream

A freshly prepared vegetarian delight.

<10 min prep

+ Add to Cart

₹50.00

LUNCH

Paneer Butter Masala

A freshly prepared vegetarian delight.

<15 min prep

+ Add to Cart

₹78.00

SNACKS

Samosa

A freshly prepared vegetarian delight.

<10 min prep

+ Add to Cart

₹30.00

LUNCH

Sizzler

A freshly prepared vegetarian delight.

<30 min prep

+ Add to Cart

₹200.00

BEVERAGES

Cold Coffee

A freshly prepared vegetarian delight.

<5 min prep

+ Add to Cart

₹90.00

BREAKFAST

Salad

A freshly prepared vegetarian delight.

<5 min prep

+ Add to Cart

₹60.00

LUNCH

Paneer Tikka

A freshly prepared vegetarian delight.

<15 min prep

+ Add to Cart

₹100.00

OTHER

Vegetable

A freshly prepared vegetarian delight.

<10 min prep

+ Add to Cart

₹70.00

BREAKFAST

LOGO

LOGO OF ANNAPURNA-SMART-CANTEEN

<10 min prep

+ Add to Cart

₹700.00

13.4 Oredring Page


Your Cart
+ items
×


Estimated delivery: 18–23 mins

	Ice Tee ₹100.00 each Remove	₹100.00  1 +
	Manchow Soup ₹45.00 each Remove	₹45.00  1 +
	Butter Nan ₹49.00 each Remove	₹49.00  1 +
	Idli (3 pices) ₹40.00 each Remove	₹40.00  1 +

Earn **+20 pts** on this order

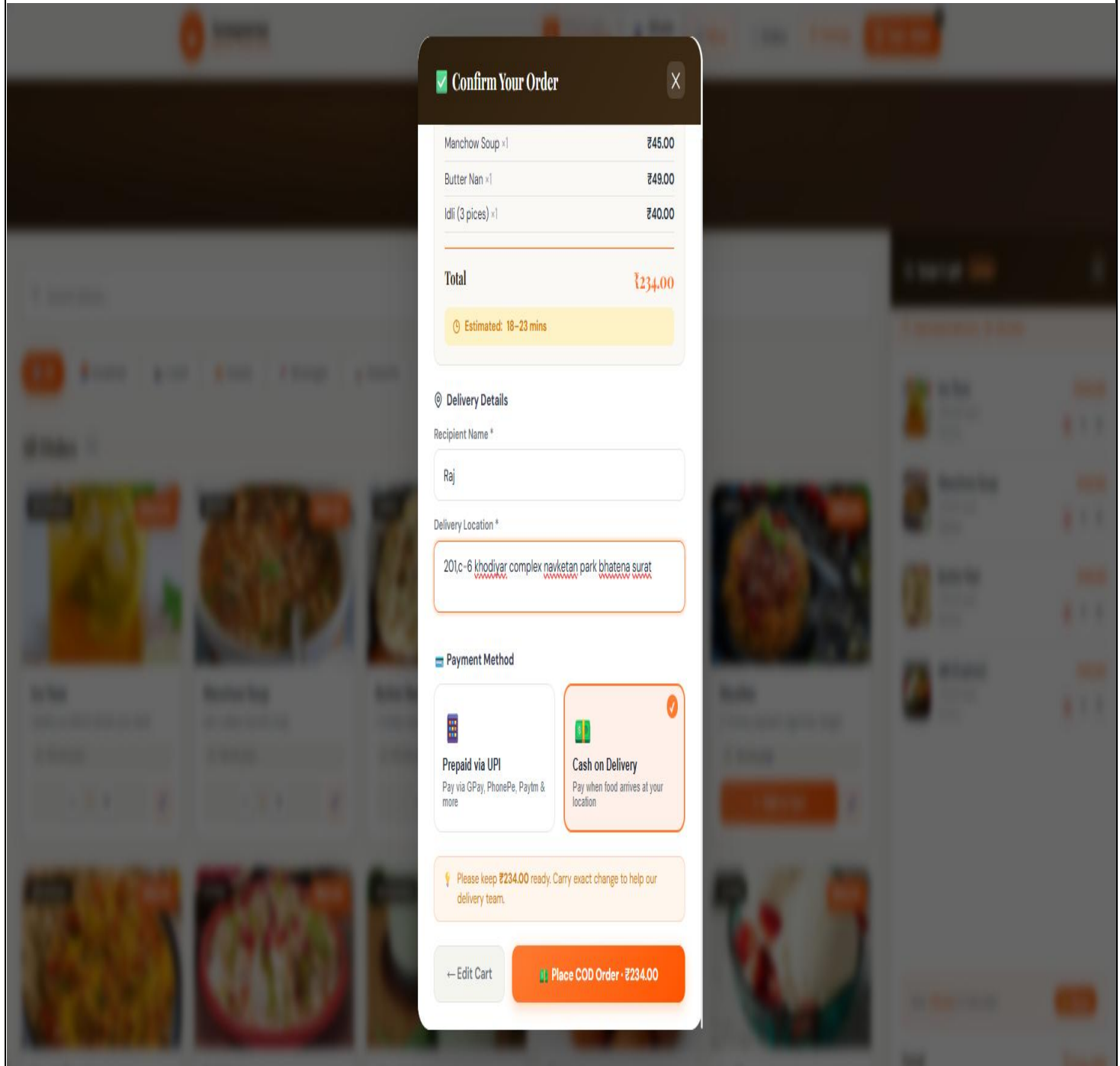

20 pts

Total
 ₹234.00


Clear entire cart


Checkout - ₹234.00

13.5 Confirm Your order Page



13.6 Succefully page

